

Training A Graph Neural Network For Botnet Detection

Presented by

Logan Flansaas

Montana Pawek



Application Context



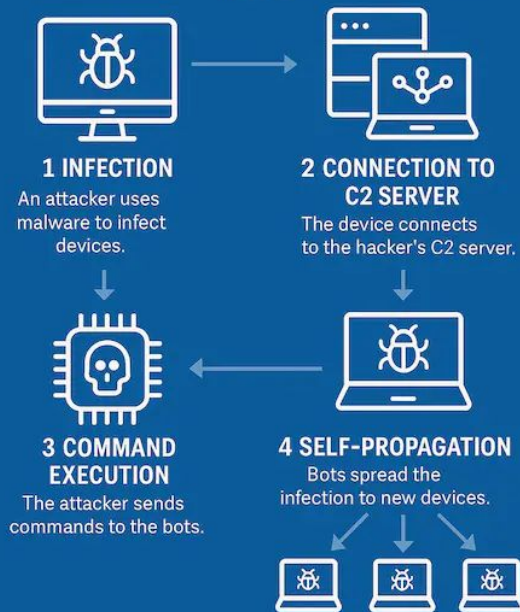
Botnets:

A coordinated network of internet-connected devices

- These devices act in unison to execute automated, large-scale cyber attacks
- Features
 - Distributed Power
 - Silent Operation
 - Automated Scalability
 - Versatility
 - Permanent Risk
- New botnets utilize automation and machine learning to exhibit decision-making abilities
- Early detection is essential to stop the spread

HOW BOTNETS WORK

A botnet works by infecting devices and controlling them via a command-and-control (C2) server.



Botnets in the World:

- Botnets can affect anyone with internet-connected devices
- In 2025 a botnet of over 5.76 million devices was detected, having grown by over 4 million that year alone [1]. The previous record was only 200,000 devices
- The number of DDoS attacks from botnets have risen by more than 50% year-over-year [2]
- Botnet detections are mainly from passive observation through security protocols, and are very hard to detect otherwise

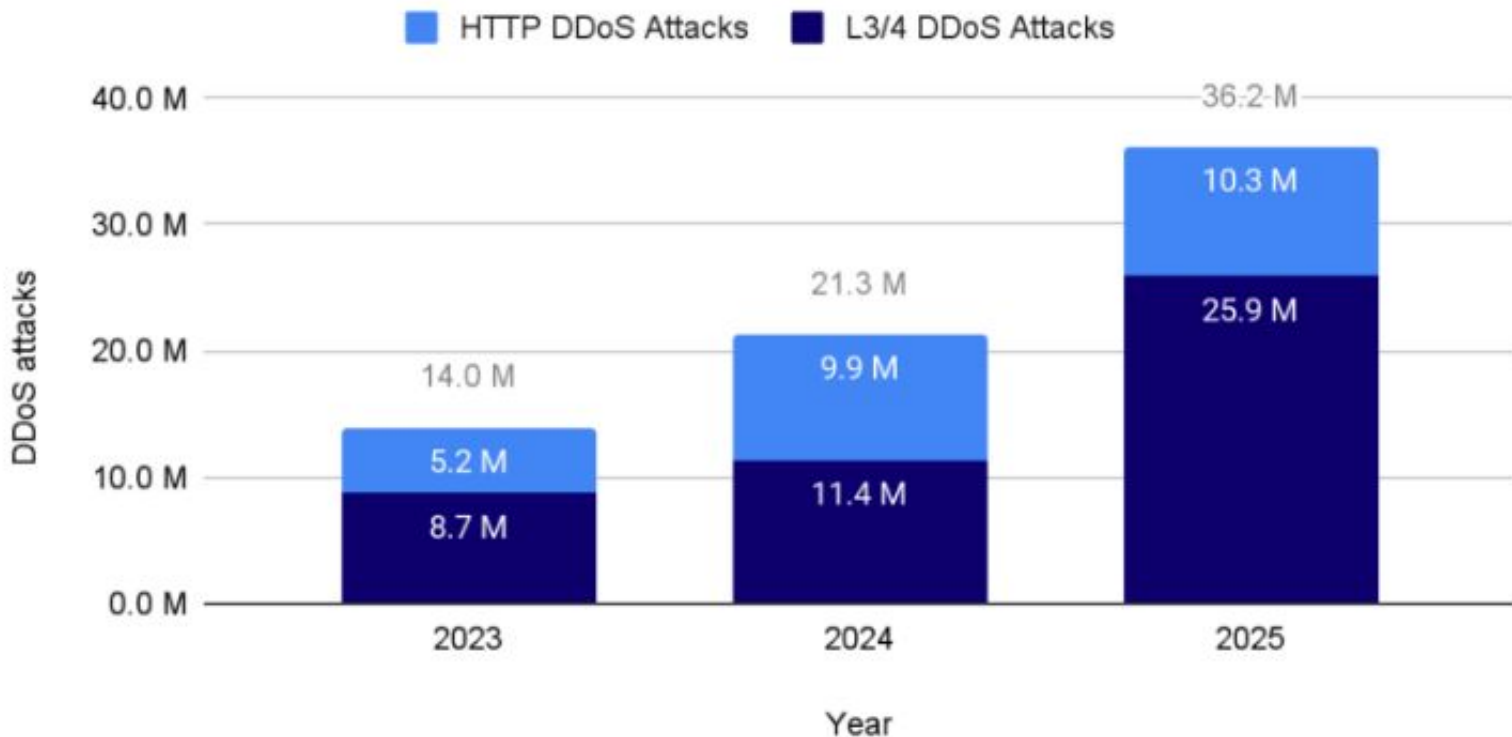
[1] Cyber Defense Wire. "Largest DDoS botnet ever recorded grew to 5.76 million devices in 2025"

[2] Cloudflare Blog. "Cloudflare's 2025 Q3 DDoS threat report"

DDoS attacks by year and type



As of October 2025



Key Technologies/Solutions

The following 6 technologies and features are essential to our model:

1. CTU-13 ☑

- CTU-13 is a labeled botnet traffic dataset built from real network communication captures
- This provides us with bidirectional NetFlow records of 13 different network scenarios with 7 different malware families

2. NetFlows ☑

- NetFlows are metadata about network traffic
- NetFlows are extracted from real world network traffic
- We create time windowed communication graphs from NetFlows
- These graphs are what our GNN operates on

3. Graphs ☑

- Nodes: an IP/host active in the time window
- Edges: communication from one host to another
- Node Features: host-level aggregated behavior
- Edge Features: communication-level aggregated behavior

Key Technologies/Solutions

4. Node Features

- In-degree / Out-degree
- Incoming / Outgoing Flow Counts
- Incoming / Outgoing Packets and Bytes
- Number of Unique Peers
- Number of Unique Ports
- Protocol Usage Counts
- Mean Duration

5. Edge Features

- Number of Flows Between Hosts
- Total Packets
- Total Bytes
- Mean Duration
- Unique Ports Used
- TCP / UDP / ICMP Counts

6. GNN Features

- GNNs are uniquely suited to identifying patterns in graphs
- GNNs learn not just from the patterns in a single node or edge, but also from a nodes neighbors
- GNNs learn from both the individual features of nodes and edges as well as their neighborhood like a CNN learns from nearby pixels

Results - Horizontal

> Traditional Methods

Intrusion Detection Systems [1]

- Mainly use signature-based detection and anomaly-based detection
- Highly efficient and reliable at detecting known threats, but generalizes poorly
- Limited adaptability
- High false positives
- Do not capture complex or coordinated behavior, only use node features

FI: ~85%

> Machine Learning

Random Forest, SVM, Logistic Regression or KNN [2]

- Use engineered network traffic features
- Strong overall performance on benchmark datasets and structured data
- Samples are treated independently, and these methods can miss relational/coordinated attack patterns, can use both node and some edge features

FI: ~97-99%*

> GNNs

Graph Neural Nets [3]

- Model a network as a graph
- Capture communication structure and coordination which can detect stealthy botnets
- Better for complex and distributed attacks which other methods struggle with, works very well with node and edge features
- Graph construction is critical

FI: ~90-97%*

[1] Arxiv. "An Anomaly-based Botnet Detection Approach for Identifying Stealthy Botnets"

[2] ResearchGate. "A Comparison of Machine Learning Approaches to Detect Botnet Traffic"

[3] Arxiv. "Automating Botnet Detection With Graph Neural Networks"

Statistics

Total Nodes:

7.5
million

Total Edges:

7.9
million

Per 5-second window:

Nodes:
4734

Edges:
4962

STD Size Dev:
~5K

AVG MB: 0.91
Max: ~6.1

Total labeled nodes:

0.13%

Positive labeled-node rate:

18.46%

Model Complexity/Parameters

Hidden Dim:

32

Input Features:

24

Layers:

2 GCN + 1
Linear

Memory:

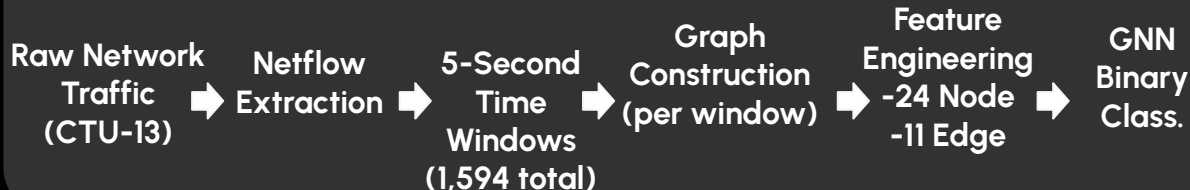
~7.5 KB

Total

Parameters:
1922

Extremely lightweight, very
scalable

End-to-End Pipeline



Results - Vertical

1

2

3

4

5

Baseline GNN

- Standard GCN using node features and graph structure
- Many false negatives
- Precision: 0.83
- Recall: 0.49

F1: ~0.60-0.62

Hyperparameters

- Tuned:
 - Hidden Dimensions
 - Dropout
 - LR
 - Weight Decay
 - Patience
- Precision: 0.62
- Recall: 0.8

F1: ~0.69-0.70

Added Threshold

- Adjusted classification threshold
- Trades precision vs recall to handle imbalances
- Precision: 0.67
- Recall: 0.87

F1: ~0.77

Edge-Aware GNN

- Added edge features
- Results were surprisingly bad likely due to extra noise
- Further tuning is needed
- Precision: 0.34
- Recall: 0.75

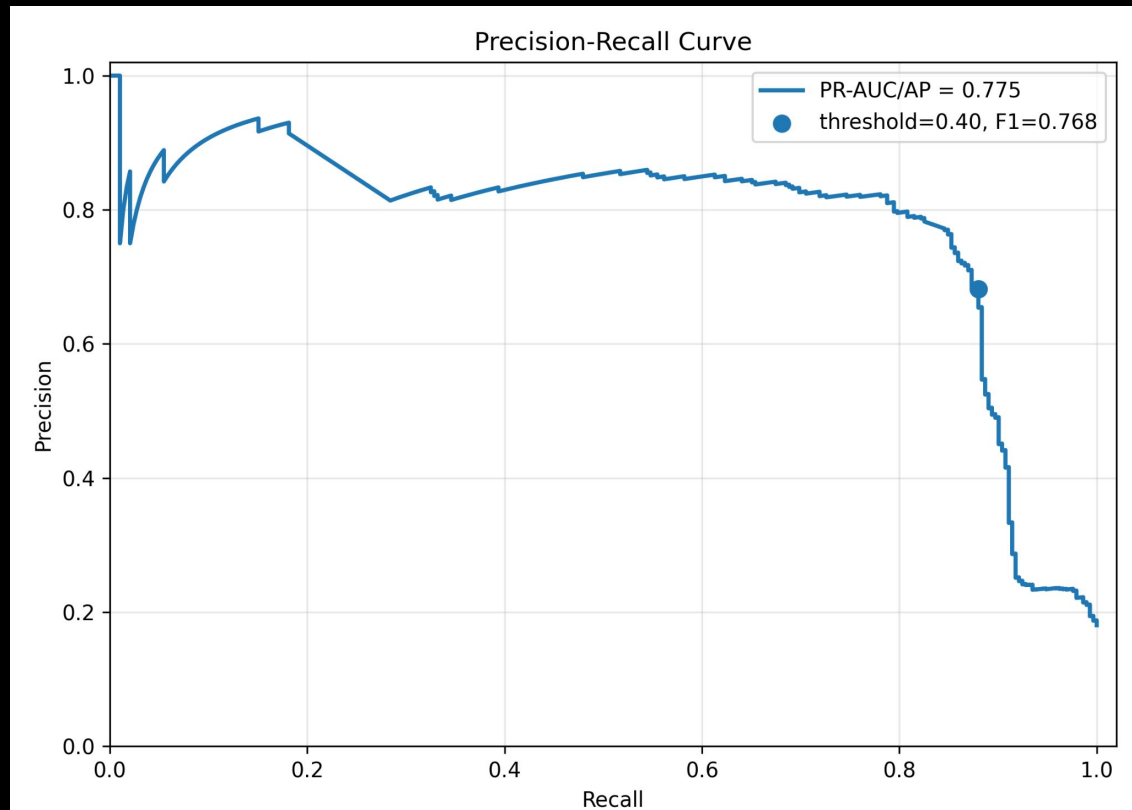
F1: ~0.47

Deeper GNN

- Extra hidden layer
- Model views relationships one node further
- Currently in testing
- Preliminary results not promising

F1: ~0.56

Visualized Results



F1 Performance Scores

0.0

1.0

Baseline GNN

0.61

Tuned Hyperparameters

0.69

Thresholded GNN

0.77

Edge-Aware GNN

0.47

Deep GNN

0.56

Thresholded GNN PR Curve:

The model retains high precision across a wide recall range despite severe class imbalance and data variance

Important Code Walkthrough

Structure of a GNN:

- Similar structure to CNN with convolutional layers
- Relu and Dropout implemented after each layer
- Linear layer is a binary classifier

```
# Initial GNN model with no added features
class SimpleGCN(nn.Module):
    def __init__(
        self,
        in_channels: int,
        hidden_dim: int,
        out_channels: int,
        dropout: float = 0.3,
    ):
        super().__init__()
        # Dropout for regularization
        self.dropout = dropout

        # Conv layers similar to a CNN
        self.conv1 = GCNConv(in_channels, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, hidden_dim)
        self.classifier = nn.Linear(hidden_dim, out_channels)

    def forward(
        self,
        x: torch.Tensor,
        edge_index: torch.Tensor,
    ) -> torch.Tensor:
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)

        logits = self.classifier(x)
        return logits
```

Moving Forward

- Find larger dataset
- Fine-tune edge feature model
- Experiment with deeper GNN